

Agentic, Offline, and Private: A Local RAG–SQL System for University Data Management

A Project Report

Submitted in partial fulfilment of the requirements for the degree of

Bachelor of Engineering
in
Computer Science and Engineering

By

Mr. Om Gawande
Mr. Yash Garad
Mr. Harshal Ninawe
Ms. Aanchal Kalambe
Ms. Aditi Parshivanikar

Under the guidance of

Dr. Vikas Bhowate

Department of Computer Science and Engineering

(Institute name)

Academic Year 2025–2026

CERTIFICATE

This is to certify that the project report entitled “**Agentic, Offline, and Private: A Local RAG–SQL System for University Data Management**” is a bona fide record of the work carried out by

Mr. Om Gawande _____

Mr. Yash Garad _____

Mr. Harshal Ninawe _____

Ms. Aanchal Kalambe _____

Ms. Aditi Parshivanikar _____

under my supervision, in partial fulfilment of the requirements for the degree of Bachelor of Engineering in Computer Science and Engineering, for the academic year 2025–2026. The work presented in this report has not been submitted elsewhere for the award of any other degree or diploma.

Dr. Vikas Bhowate
Project Guide

Head of Department
Computer Science and Engineering

Place: _____

Date: _____

DECLARATION

We hereby declare that the project entitled “**Agentic, Offline, and Private: A Local RAG–SQL System for University Data Management**” is an original piece of work carried out by us under the guidance of Dr. Vikas Bhowate, and that it has not been submitted, in whole or in part, for the award of any other degree, diploma, or similar title at this or any other institution. All sources of information used in this report have been duly acknowledged.

Mr. Om Gawande

Mr. Yash Garad

Mr. Harshal Ninawe

Ms. Aanchal Kalambe

Ms. Aditi Parshivanikar

Date: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. Vikas Bhowate, for his continual guidance, technical insight, and encouragement throughout the design, implementation, and evaluation of this project. His emphasis on rigorous, verifiable engineering — rather than superficial demonstration — shaped both the architecture of the system described in this report and the discipline with which it was tested.

We also thank the Department of Computer Science and Engineering for providing the infrastructure and environment necessary to carry out this work, and our peers for their feedback during informal reviews of the system.

Finally, we acknowledge the open-source communities behind Django, React, PostgreSQL, Qdrant, and Ollama, whose freely available tools made a fully offline, self-hosted deployment of this scope achievable within an academic project timeline.

ABSTRACT

Institutions that want to let students and staff ask natural-language questions about courses, faculty, fees, and schedules face an uncomfortable choice: route those questions through a cloud large-language-model (LLM) API and accept that institutional and student data leaves the campus network, or forgo conversational access altogether. This project presents *College Assistant*, an agentic, fully offline question-answering system that resolves this trade-off by running every stage of inference — retrieval, SQL generation, answer synthesis, and answer verification — on institution-controlled hardware, using open-weight models served locally by Ollama.

The system decomposes what would otherwise be a single, over-trusted LLM call into five cooperating agents: a router that classifies each question, a SQL agent that generates and executes strictly read-only, schema-restricted queries against the live institutional database, a retrieval-augmented generation (RAG) agent that searches a locally hosted Qdrant vector store for descriptive context, a synthesis agent that composes the final answer, and a verification agent that checks the answer against its evidence before it is returned. A dedicated web-fetch agent may consult a fixed, keyword-matched allowlist of official institutional pages; the model is never permitted to choose a URL. Generated SQL is independently parsed and validated by a guard built on `sqlglot` before execution, and the database role available to the model carries no write privileges and no access to personally identifiable student records, so a fully compromised or adversarially prompted model still cannot exfiltrate credentials, modify data, or read another user’s conversation.

The system was evaluated on commodity, GPU-less hardware (16 CPU cores, no CUDA accelerator). Targeted optimisation of the routing and verification stages reduced router latency by roughly four orders of magnitude (5.7s to 0.4ms) and cut short-answer verification latency from 11.3s to 50ms, bringing end-to-end median response time for a seven-question benchmark down from 1,455.8s to 876.7s while holding routing accuracy at 7/7 identical routes. A follow-on scaling study identified model eviction between requests as the single largest latency contributor (a 19.6× slowdown when the LLM had to reload) and showed that a semantic answer cache alone does nothing for simultaneous identical questions — request coalescing was required to take a 50-user, 5-question workload from 2 served requests per minute to all 50 succeeding at 28.1 requests/min.

The resulting system demonstrates that a multi-agent, defence-in-depth architecture can deliver natural-language access to live institutional data without a cloud dependency, without granting the language model direct write or broad read access, and at a latency that, after optimisation, is usable on hardware a mid-sized institution can reasonably own outright.

Keywords: Retrieval-Augmented Generation, Text-to-SQL, Multi-Agent Systems, Local LLM Inference, Data Privacy, University Information Systems, Ollama, Vector Search.

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	Problem Statement	1
1.3	Objectives	1
1.4	Scope	2
1.5	Organisation of the Report	2
2	Literature Survey	3
2.1	Retrieval-Augmented Generation	3
2.2	Text-to-SQL	3
2.3	Local and Open-Weight LLM Serving	3
2.4	Multi-Agent and Tool-Using LLM Architectures	3
2.5	Positioning of This Work	4
3	System Requirements and Analysis	5
3.1	Functional Requirements	5
3.2	Non-Functional Requirements	5
3.3	Hardware and Software Requirements	6
3.4	Feasibility Study	6
4	System Design and Architecture	7
4.1	High-Level Architecture	7
4.2	Agent Pipeline and Request Lifecycle	7
4.3	Data Layer	8
4.4	Security Architecture	8
4.5	Deployment Architecture	9
5	Implementation	10
5.1	Technology Stack	10
5.2	Orchestrator	10
5.3	Router Agent	10
5.4	SQL Agent and Guard	11
5.5	RAG Agent	11
5.6	Synthesis and Verification Agents	11
5.7	Web Agent	11
5.8	Frontend	11
5.9	Authentication and Session Management	12
5.10	Representative Code Excerpt	12
6	Testing	13
6.1	Testing Strategy	13
6.2	Unit Test Coverage	13
6.3	Security Testing	13
6.4	Representative Test Cases	14
7	Results and Performance Evaluation	15
7.1	Experimental Setup	15
7.2	Latency Optimisation	15
7.3	Where the Latency Actually Goes	15
7.4	Model Eviction and Concurrency Findings	16
7.5	Discussion	16
7.6	Limitations	17
8	Conclusion and Future Scope	18
8.1	Conclusion	18
8.2	Future Scope	18

Chapter 1

Introduction

1.1 Background and Motivation

Universities and colleges hold most of their day-to-day useful information in two very different forms. Structured, transactional facts — which course runs in which room, what a subject’s credit weight is, what the last date to pay a fee instalment is — live in a relational database behind an administrative portal. Descriptive, slow-changing information — what a department does, how a scholarship scheme works, what a policy says — lives in prose, spread across handbooks, circulars, and web pages. A student or member of staff with a question rarely knows, or should have to know, which of these two worlds their question belongs to; they simply want an answer.

Large language models make a conversational interface to this kind of mixed knowledge base technically straightforward to prototype: point a chatbot at the database, let the model write the query, and let it read the handbook too. It is exactly this convenience that makes the naive version of the idea unsafe to deploy. Sending student questions — and, if the assistant is any good, fragments of student records in the answers — to a third-party cloud API moves personal data outside institutional control, which sits uneasily against data-protection expectations and, in India, the obligations institutions carry under the Digital Personal Data Protection Act, 2023. Separately, and just as seriously, handing a language model the ability to generate and directly execute arbitrary SQL against a production database turns every prompt-injection vulnerability into a potential data breach or data-loss incident, because there is nothing between the model’s output and the database engine.

This project is an attempt to keep the convenience of a conversational interface while removing both of those exposures: keep every model and every byte of institutional data on infrastructure the institution controls, and never let the model’s output reach the database, the file system, or the public internet without an independent, non-LLM check in between.

1.2 Problem Statement

To design, implement, and evaluate a question-answering system for university data that:

1. answers natural-language questions drawing on both structured (database) and unstructured (descriptive) institutional knowledge,
2. performs all language-model inference locally, so that no student data, question, or answer is transmitted to a third-party service,
3. restricts the language model to read-only, schema-limited database access validated by an independent guard, so that a fully compromised or adversarially prompted model cannot write to, or exfiltrate unauthorised data from, the database, and
4. remains usable — in terms of response latency — when deployed on commodity server hardware without a dedicated GPU accelerator.

1.3 Objectives

1. Provide a single chat interface that transparently answers both record-lookup questions (“When is the mid-semester exam for Computer Networks?”) and descriptive questions

(“What is the process for a branch-change request?”).

2. Run all LLM and embedding inference locally through Ollama, with no external API dependency at query time.
3. Enforce that the language model can only reach the database through a role with no write privileges and no access to personally identifiable student data, and that every generated query is independently validated before execution.
4. Decompose the assistant into single-purpose agents (routing, retrieval, SQL execution, synthesis, verification, web lookup) so that the compromise or malfunction of one agent has a bounded, well-defined blast radius rather than compromising the whole system.
5. Provide institution-appropriate authentication, rate limiting, session management, and audit logging.
6. Measure and reduce end-to-end response latency on CPU-only hardware to a level usable in a live chat interface, and characterise the system’s behaviour under concurrent multi-user load.

1.4 Scope

The system covers read access to institutional data: courses, departments, faculty, fee schedules, timetables, and general institutional policy documents. It deliberately excludes write operations of any kind through the AI pipeline — admissions processing, grade entry, and fee collection remain the responsibility of existing administrative systems and are out of scope. Multi-institution deployment, a native mobile client, and GPU-accelerated serving are identified as future work in Chapter 8 rather than implemented in the present system.

1.5 Organisation of the Report

Chapter 2 surveys related work in retrieval-augmented generation, text-to-SQL, and multi-agent LLM systems, and positions this project against it. Chapter 3 analyses functional and non-functional requirements and feasibility. Chapter 4 presents the system architecture. Chapter 5 describes the implementation of each component. Chapter 6 covers the testing strategy and results. Chapter 7 presents performance evaluation and optimisation results measured on the running system. Chapter 8 concludes and outlines future work.

Chapter 2

Literature Survey

2.1 Retrieval-Augmented Generation

Lewis et al. [1] introduced retrieval-augmented generation (RAG) as a way to ground a language model’s output in passages retrieved from an external knowledge source at inference time, rather than relying solely on facts memorised during training. This reduces hallucination and allows the knowledge base to be updated without retraining the model — a property this project relies on directly: institutional policy documents change far more often than any model could reasonably be retrained. The system described in this report follows the now-standard RAG pattern of embedding a query, retrieving the nearest passages from a vector index, and conditioning generation on the retrieved text, but applies it only to the descriptive half of the knowledge base, keeping structured data on a separate, SQL-based path.

2.2 Text-to-SQL

Yu et al. [2] released Spider, a large cross-domain benchmark for translating natural-language questions into SQL, which established text-to-SQL as a distinct and difficult sub-problem of semantic parsing: the model must reason jointly about the question and an unfamiliar database schema. The literature on text-to-SQL is largely concerned with translation *accuracy*; comparatively little of it addresses what happens when the generated SQL is wrong, or adversarially manipulated, and is executed directly against a live database. This project treats that as a first-class design concern rather than an afterthought: generated SQL is never executed as-is, but is parsed and checked by an independent guard (Section 5.4) before it reaches the database, and the database connection it runs under is restricted at the role level regardless of what the guard catches.

2.3 Local and Open-Weight LLM Serving

The release of open-weight model families such as LLaMA [4] and the broader ecosystem of tools built to serve such models on commodity hardware (Ollama being the one used in this project) has made on-premise LLM deployment practical without the recurring per-token cost, or the data-egress requirement, of a hosted API. This project uses the Qwen2.5 open-weight model family [5] for routing, SQL generation, answer synthesis, and verification, and the `nomic-embed-text` model for embeddings, all served locally by Ollama with no outbound network access required at query time.

2.4 Multi-Agent and Tool-Using LLM Architectures

Yao et al. [3] showed that interleaving reasoning traces with discrete tool actions (the ReAct pattern) improves reliability over a single free-form generation for tasks that require interacting with an external environment. This project takes a related but more constrained approach: rather than one agent freely choosing which tools to call, responsibilities are statically partitioned across five specialised agents (Section 4.2), each with a narrow, independently auditable interface. This trades some of the flexibility of a fully autonomous tool-using agent for a much smaller and more analysable attack surface, which was judged appropriate for a system with access to institutional data.

2.5 Positioning of This Work

Table 2.1 compares three deployment patterns for a university question-answering assistant: a cloud-hosted chatbot wired directly to institutional systems, a purely descriptive (FAQ-only) local RAG bot, and the system built in this project.

Table 2.1: Comparison of university assistant deployment patterns

Property	Cloud chatbot + live DB	Local RAG, FAQ only	This system
Data leaves campus	Yes, every query	No	No
Answers live structured data	Yes, directly	No	Yes, via guarded read-only SQL
SQL independently validated	Rarely	N/A	Yes (sqlglot guard)
Blast radius of a compromised model	Whole database	Minimal (no DB access)	12 non-sensitive tables, read-only
Recurring per-query cost	Yes (API)	No	No
Requires GPU hardware	No (cloud-side)	Usually not	No (optimised for CPU)

The gap this project addresses is that, in practice, deployed university assistants tend to sit at one of the two extremes in Table 2.1: either they cannot answer questions that require live structured data, or they can, but without an independent safety layer between the model and the database. This project’s contribution is an architecture that answers both kinds of question from a single interface while keeping the safety guarantees of the FAQ-only design.

Chapter 3

System Requirements and Analysis

3.1 Functional Requirements

Table 3.1: Functional requirements

ID	Requirement
FR1	Users authenticate with an institutional email address and a verified one-time password before accessing the assistant.
FR2	A user may submit a natural-language question through a chat interface.
FR3	The system classifies each question and routes it to a SQL lookup, a descriptive (RAG) lookup, both, or a web lookup.
FR4	Structured questions are answered by generating and executing a read-only SQL query against the live institutional database.
FR5	Descriptive questions are answered from a locally indexed corpus of institutional documents using vector similarity search.
FR6	When a question needs both sources, the system merges structured and descriptive evidence into one answer.
FR7	Every generated answer is checked against its supporting evidence by an independent verification step before it is returned to the user.
FR8	Answers are streamed to the client token-by-token rather than returned only on completion.
FR9	Conversation history is persisted per authenticated user.
FR10	Administrative and operational staff can view system health and audit logs.

3.2 Non-Functional Requirements

- **Data locality.** No student question, answer, or database content may be transmitted to a service outside institution-controlled infrastructure.
- **Least privilege.** The database role used by the language model must have no write privileges and no grant on any table containing personally identifiable student data.
- **Usable latency.** Time to first token should be minimised on CPU-only hardware; Chapter 7 documents the optimisation carried out against this requirement.
- **Session security.** Sessions must expire after a bounded idle period, and repeated failed logins against one account must be throttled independently of the source IP address.
- **Auditability.** Security-relevant events (logins, lockouts, web fetches, rejected SQL) must be logged.
- **Horizontal headroom.** The system should degrade gracefully, not collapse, under concurrent load from many simultaneous users asking a small number of common questions.

3.3 Hardware and Software Requirements

Table 3.2: Deployment requirements

Category	Requirement
Server hardware	16 GB RAM minimum (32 GB recommended so all three language models can stay resident); 20 GB free disk; CPU-only operation supported, CUDA GPU optional and beneficial.
Containerisation	Docker Desktop / Docker Engine with Compose.
Backend	Python 3.12, Django, Django REST Framework.
Frontend	Node.js / Vite build toolchain, React.
Data stores	PostgreSQL (relational), Qdrant (vector).
LLM runtime	Ollama, serving <code>qwen2.5:7b</code> and <code>nomic-embed-text</code> .
Reverse proxy / TLS	Caddy (local certificate authority for development, automatic HTTPS in production).

3.4 Feasibility Study

3.4.1 Technical Feasibility

Every component — Django, PostgreSQL, Qdrant, Ollama, React, Caddy — is open source and self-hostable, so the architecture in Chapter 4 does not depend on any proprietary or cloud-only service. The main technical risk identified up front was inference latency on CPU-only hardware, which Chapter 7 shows was reduced to a usable level through targeted optimisation rather than requiring a GPU.

3.4.2 Economic Feasibility

Because inference runs locally, the system carries no recurring per-query API cost. The only ongoing cost is the electricity and depreciation of institution-owned server hardware, which compares favourably against a cloud LLM API billed per token at the volume a mid-sized institution’s question traffic would generate.

3.4.3 Operational Feasibility

The entire stack is defined as a single Docker Compose file and is brought up with one command, with health checks exposed for each service (Section 4.5). This keeps the operational burden on institutional IT staff comparable to running any other self-hosted web application.

Chapter 4

System Design and Architecture

4.1 High-Level Architecture

Figure 4.1 shows the layered architecture of the system: a React chat client, a Django REST Framework API gateway handling authentication and request validation, a backend services layer containing the five agents described in Section 4.2, an external services layer for the local LLM runtime, and a data layer holding the relational database, the vector store, and the change-sync worker that keeps them consistent.

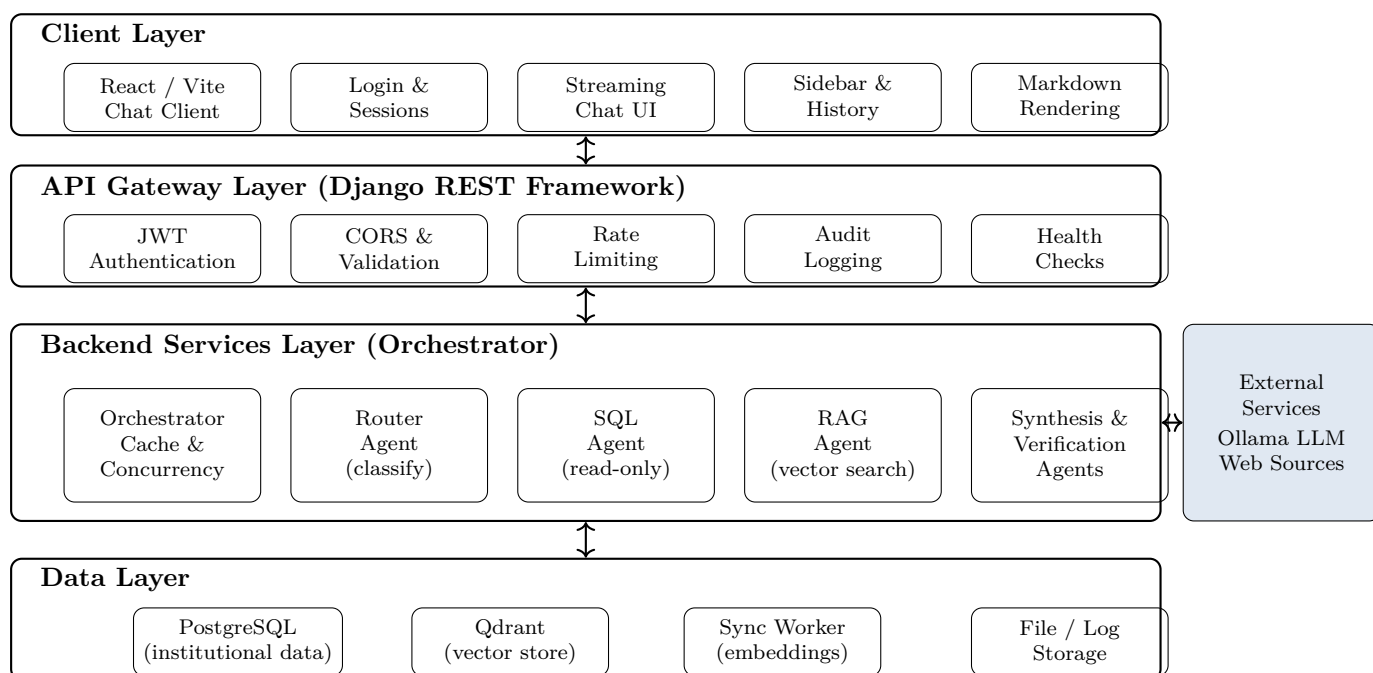


Figure 4.1: Layered system architecture

4.2 Agent Pipeline and Request Lifecycle

Figure 4.2 traces a single question through the system. The router first classifies the question into one of four routes — SQL, RAG, BOTH, or WEB — using a fast, rule-and-embedding-based classifier (Section 5.3) rather than a full LLM call. Depending on the route, the SQL agent, the RAG agent, or the web-fetch agent (or a combination) gathers evidence; the synthesis agent then composes a natural-language answer from that evidence and streams it to the client token-by-token; and the verification agent independently checks the streamed answer against the evidence it was given before the response is marked complete. If a downstream source is unavailable, the orchestrator degrades gracefully and annotates the answer rather than failing the request outright (Section 5.2).

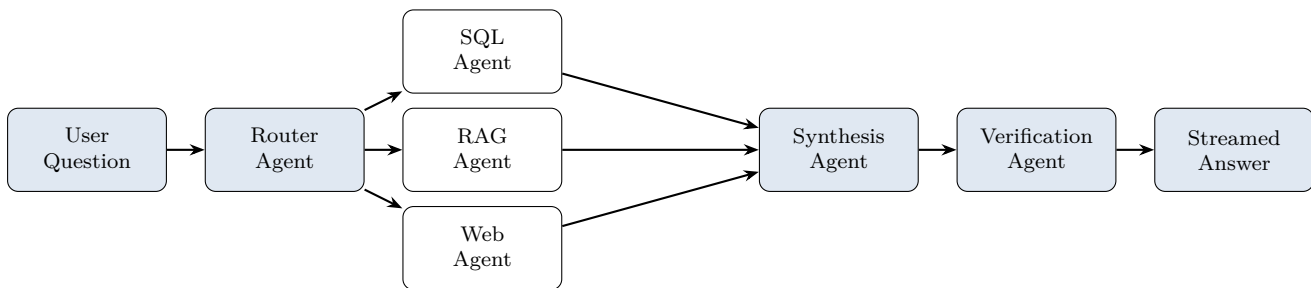


Figure 4.2: Agent pipeline for a single request

4.3 Data Layer

Structured institutional data lives in PostgreSQL. The language model reaches it only through a dedicated read-only role, `rag_agent_ro`, granted `SELECT` on 12 named tables covering courses, departments, faculty, fee schedules, and timetables; tables holding per-student records (`students`, `enrollments`, `exam_results`, `fee_payments`, `attendance`) and tables holding credentials or conversation history are not granted to this role at all, so they are unreachable through the AI pipeline regardless of what SQL the model generates (Section 5.4 and Section 4.4 detail the enforcement).

Descriptive content — institutional documents, policies, and other free-text material — is embedded with `nomic-embed-text` and stored in a Qdrant collection. A background sync worker watches a change queue on the relational database, converts changed rows to text, generates embeddings, and upserts them into Qdrant, so the vector index stays consistent with the system of record without requiring a full reindex on every update.

4.4 Security Architecture

Three controls hold even if the language model itself is fully subverted by a prompt-injection attack, because they do not depend on the model behaving correctly:

1. **Read-only, schema-restricted database role.** Every SQL statement the model causes to run executes under `rag_agent_ro`, which has `default_transaction_read_only` set at the role level (so writes are blocked before privileges are even checked), is granted `SELECT` on 12 non-sensitive tables only, and carries `NOSUPERUSER` `NOCREATEDB` `NOCREATEROLE` and a connection limit of 10.
2. **Independent SQL validation.** Generated SQL is parsed with `sqlglot` and rejected unless it is a single, capped `SELECT` statement referencing only allow-listed tables, with results capped at 50 rows. The check is token-level rather than regex-based, so a course literally titled “Update Systems” is not mistaken for an `UPDATE` statement.
3. **No model-chosen URLs.** The web-fetch agent selects from a fixed `urls_allowlist.json` by deterministic keyword matching. The model is never asked for, and cannot supply, a URL. Matching is exact-string (so a look-alike domain is rejected), redirects are followed manually with every hop re-checked, and DNS results are checked so an allow-listed host-name resolving to a private or link-local address — including the cloud metadata address `169.254.169.254` — is refused.

Around these three structural controls sits a conventional application security layer: PBKDF2-hashed passwords with a 12-character minimum and common-password rejection, a 30-minute idle session timeout, per-user and per-IP rate limiting, and account lockout after five failed attempts, described further in Section 5.9.

4.5 Deployment Architecture

The system is defined as a single Docker Compose stack with the services listed in Table 4.1. Each service exposes a health-check endpoint, and the backend will not report healthy until its dependencies (database, vector store, LLM runtime) are themselves reachable.

Table 4.1: Docker Compose services

Service	Role
postgres	Relational database holding institutional data and application state.
qdrant	Vector database for descriptive-content retrieval.
ollama / ollama-pull	Local LLM runtime; ollama-pull fetches the configured models on first start.
backend	Django REST Framework API, orchestrator, and all five agents.
sync_worker	Keeps the Qdrant index consistent with PostgreSQL.
caddy	Reverse proxy issuing local (development) or automatic (production) TLS certificates.

Chapter 5

Implementation

5.1 Technology Stack

Table 5.1: Technology stack

Layer	Technology
Backend framework	Python 3.12, Django, Django REST Framework
Frontend	React, Vite, Tailwind CSS
Relational database	PostgreSQL
Vector database	Qdrant
LLM runtime	Ollama, serving <code>qwen2.5:7b</code> (generation) and <code>nomic-embed-text</code> (embeddings)
SQL validation	<code>sqlglot</code>
Auth & sessions	Django’s PBKDF2-SHA256 hashing, JWT, DRF throttling
Reverse proxy / TLS	Caddy
Containerisation	Docker, Docker Compose

5.2 Orchestrator

The orchestrator (`orchestrator/service.py`) is the entry point for every question. It resolves a route by calling the router agent, dispatches to the SQL, RAG, and/or web agents concurrently using a thread pool, applies a concurrency guard (`llm_slot`) so the number of simultaneous LLM calls stays within what the resident models can serve, checks a semantic cache before generating, and appends a degradation note to the answer — for example, *“live records lookup is temporarily unavailable, so this answer is based on descriptive information only”* — when one evidence source is down but the system can still answer from the other rather than failing the whole request.

5.3 Router Agent

The router agent (`router_agent/classifier.py`) decides between the SQL, RAG, BOTH, and WEB routes. The original implementation asked a 7-billion-parameter model to classify each question from a roughly 1,200-token few-shot prompt, which Chapter 7 shows cost 5.7 seconds per question for a decision worth about twenty tokens of JSON. It was replaced with a rule-and-embedding based classifier that makes the same decision in well under a millisecond with no measured change in routing accuracy on a 29-question labelled evaluation set.

5.4 SQL Agent and Guard

The SQL agent (`sql_agent/service.py`) prompts the LLM with the allow-listed schema and the user's question, and receives a candidate SQL statement back. That statement is never executed directly. It first passes through `sql_agent/guard.py`, which:

- parses the statement with `sqlglot` rather than matching it against a regular expression, so that a string literal containing a keyword like `UPDATE` is not mistaken for the keyword itself;
- rejects anything that is not a single `SELECT` statement, any table outside the allow-list, and any of the forbidden keyword tokens `INSERT`, `UPDATE`, `DELETE`, `DROP`, and `ALTER`;
- caps the result set at 50 rows.

Only a statement that survives the guard is executed, and it is executed under the read-only `rag_agent_ro` role described in Section 4.4, so even a guard bypass would still be bounded by what that role can read.

5.5 RAG Agent

The RAG agent (`rag_agent/service.py`, `rag_agent/vector_store.py`) embeds the incoming question with `nomic-embed-text` and retrieves the top- k ($k = 5$) nearest passages from the Qdrant collection populated by the sync worker. Retrieved passages are passed to the synthesis agent as evidence rather than being shown to the user directly.

5.6 Synthesis and Verification Agents

The synthesis agent (`synthesis_agent/service.py`) composes the final answer from whatever evidence the SQL, RAG, and/or web agents produced, streaming tokens to the client as they are generated (`synthesize_answer_stream`). The verification agent then checks the completed answer against the same evidence before the response is marked final. For short, low-claim-count answers (typically SQL-sourced), a no-LLM fast path performs this check directly, which Chapter 7 shows cut verification latency for that case from 11.3 seconds to 50 milliseconds; for longer, evidence-dense answers a terse-verdict LLM call is still used, but a more compact verdict format alone brought RAG-answer verification down from 168.5 seconds to 62.3 seconds.

5.7 Web Agent

The web agent (`web_agent/service.py`) is consulted only for questions matched by keyword to a fixed `urls_allowlist.json` of official institutional pages. As noted in Section 4.4, the model never supplies a URL; the agent selects one deterministically. Fourteen tests in `web_agent/tests.py` cover this path, including live refusal of internal service addresses (`ollama:11434`, `postgres:5432`, `qdrant:6333`), the cloud metadata address, and `file://` URIs.

5.8 Frontend

The client is a React application built with Vite, using Tailwind CSS for styling and `react-markdown` with `rehype-highlight` to render streamed answers, including code blocks, as they arrive. It authenticates against the Django backend, maintains conversation history per user, and renders the same degradation notes the orchestrator attaches when an evidence source was unavailable, so the limitation is visible to the user rather than silently hidden.

5.9 Authentication and Session Management

Authentication combines an institutional email/password login with JWT-based API sessions. Passwords are hashed with Django's default PBKDF2- SHA256, must be at least 12 characters, and are checked against a list of roughly 20,000 common passwords and against similarity to the user's own name, username, and email. Sessions expire after 30 minutes of inactivity and on browser close; session cookies are `HttpOnly`, `Secure`, and `SameSite=Lax`. Login is rate-limited to 10 attempts per minute per IP address, and, independently, an account is locked for 15 minutes after five consecutive failed attempts against it — a control that matters because it bounds guesses against one account regardless of how many source IP addresses an attacker distributes the attempt across, which a purely per-IP limit cannot do. During the security review documented in Chapter 6, the login endpoint was found to be exempt from CSRF protection (a consequence of Django REST Framework marking `@api_view` views `csrf_exempt` by default) and was fixed before deployment.

5.10 Representative Code Excerpt

Listing 5.1 shows the route-resolution function from the orchestrator: it calls the router agent, falls back to answering from both sources if routing itself fails, and returns both the chosen route and a human-readable reason that is logged for later audit.

Listing 5.1: Route resolution in the orchestrator (`orchestrator/service.py`)

```
1 def _resolve_route(question):
2     # classify() can raise LLMUnavailable (Ollama down) -- we let that
3     # propagate, since if the LLM is down synthesis can't run either. A
4     # *parse* failure (route_result.error) is different: fall back to
5     # BOTH and carry on.
6     route_result = classify(question)
7     if route_result.error:
8         logger.warning(
9             "router failed question=%r error=%s -- falling back to %s",
10            question, route_result.error, DEFAULT_ROUTE_ON_ERROR,
11        )
12        return DEFAULT_ROUTE_ON_ERROR, (
13            f"router error, defaulted to {DEFAULT_ROUTE_ON_ERROR}"
14        )
15    return route_result.route, route_result.reason
```

Chapter 6

Testing

6.1 Testing Strategy

Testing was carried out at two levels. Unit and integration tests, run with Django’s test runner, cover each agent’s decision logic in isolation (classification, SQL guard rejections, verification verdicts) and the authentication flow. Security testing was carried out against the running system rather than only at the unit level, because the properties that matter most — can a write actually reach the database, can the web agent actually be redirected to an internal address — are only meaningfully true if demonstrated against the live stack, not merely asserted in a mock.

6.2 Unit Test Coverage

Table 6.1: Unit tests by module

Module	Test count
<code>accounts</code> (authentication, logout, sessions)	24
<code>web_agent</code> (allowlist, SSRF refusal)	24
<code>verification_agent</code>	17
<code>synthesis_agent</code>	14
<code>orchestrator</code>	13
<code>router_agent</code>	8
<code>sql_agent</code> (guard rejections)	5
Total	105

6.3 Security Testing

Several controls were verified directly against the running system rather than only through unit tests:

- **Read-only role enforcement.** `manage.py check_rag_agent_ro --check-write` attempted a real INSERT under the `rag_agent_ro` role; it was rejected with *“session is read-only”*. This was re-verified after the web-fetch agent and multi-user authentication were added, to confirm neither change widened the role’s privileges.
- **SSRF / URL restriction.** Live requests to `ollama:11434`, `postgres:5432`, `qdrant:6333`, the cloud metadata address `169.254.169.254`, and a `file:///etc/passwd` URI were all refused by the web agent.
- **Brute-force logout.** Five consecutive failed logins against one account triggered a 15-minute logout, confirmed to persist across a backend restart because logout state is stored on the database row rather than in an in-memory cache.

- **Login CSRF.** Prior to the fix described in Section 5.9, a forged cross-site POST to the login endpoint returned a valid session cookie for the attacker’s account without a CSRF token; after the fix, the same request is rejected.

6.4 Representative Test Cases

Table 6.2: Representative test cases

Test	Input / Action	Expected result
SQL guard, forbidden keyword	Model generates a statement containing <code>DROP TABLE</code>	Statement rejected before execution
SQL guard, false-positive keyword	Question about a course titled “Update Systems”	Statement accepted; <code>UPDATE</code> recognised as a string literal, not a keyword
Read-only role	Direct <code>INSERT</code> attempted as <code>rag_agent_ro</code>	Rejected: session is read-only
Web agent, look-alike domain	Allow-listed <code>example.com</code> vs. <code>example.com.attacker.test</code>	Look-alike domain rejected
Account lockout	5 failed logins against one account	6th attempt rejected regardless of source IP; lock persists across restart
Login CSRF (pre-fix)	Forged cross-site POST to <code>/api/auth/login/</code>	Valid session issued (vulnerability, since remediated)
Degraded mode	Database made unreachable mid-session	RAG-only answer returned with a note that live records were unavailable

Chapter 7

Results and Performance Evaluation

7.1 Experimental Setup

All measurements in this chapter were taken on an Intel Core Ultra 9 185H (16 cores / 22 threads), 31.4 GB RAM, with **no CUDA GPU** — a deliberately conservative choice of hardware, since it represents the lower bound an institution might actually deploy on rather than a best-case research machine. On this hardware, Ollama generated at 9.0 tokens/second and read prompts at 44 tokens/second. Per-stage timings were captured for every request by an always-on profiler (`orchestrator/profiling.py`) riding along on the response’s completion event, so the figures below come from the real API rather than a synthetic benchmark harness.

7.2 Latency Optimisation

Table 7.1 summarises the effect of two targeted changes: replacing the LLM-based router with the rule-and-embedding classifier of Section 5.3, and introducing a no-LLM fast path plus a terser verdict format for verification (Section 5.6). The same seven questions, in the same order, were run against the backend before and after the change, with the backend restarted between runs.

Table 7.1: Latency before and after optimisation

Stage	Before	After	Improvement
Router	5,742 ms	0.4 ms	$\sim 14,000\times$
Verification (SQL answers)	11,270 ms	50 ms	$225\times$
Verification (RAG answers)	168,485 ms	62,314 ms	$2.7\times$
Time to first token	79.8 s	46.8 s	$1.7\times$
End-to-end, 7 questions	1,455.8 s	876.7 s	$1.66\times$

Routing accuracy did not move: both runs produced identical routes on all seven questions, and a separate 29-question labelled evaluation set measured 100% precision at 97% coverage for the new classifier. No answer was truncated by the change; the longest answer observed was 1,252 characters against a roughly 3,500-character cap. Synthesis time itself was essentially unchanged (76.1 s to 78.5 s median) and two of the seven questions were individually slower after the change, entirely from run-to-run synthesis variance — CPU-bound generation time was observed to vary by up to $\pm 30\%$ between otherwise identical runs, so single-run synthesis figures should be read with that caveat; the router and verification improvements, changing by two to four orders of magnitude, are not subject to the same noise.

7.3 Where the Latency Actually Goes

A separate single-question profile (Table 7.2), taken with all models already resident in memory, shows that LLM inference dominates the pipeline and the database does not.

The conclusion drawn from this profile is that further database tuning has essentially nothing left to win — the database accounts for roughly 0.1% of pipeline time — and that any further latency work should target LLM inference itself, which the two findings below address.

Table 7.2: Per-stage share of latency, warm models, single SQL question

Stage	Time	Share
Verification (LLM)	5.17 s	27%
Router classify (LLM, pre-optimisation)	3.29 s	17%
SQL generate + execute (LLM)	2.72 s	14%
Synthesis stream (LLM)	2.61 s	14%
Qdrant vector search	0.18 s	1%
Database: schema introspection	0.02 s	0.1%
Database: connect	0.01 s	0.0%
Database: execute agent query	0.00 s	0.0%
Total LLM share		72.5%

7.4 Model Eviction and Concurrency Findings

7.4.1 Model residency dominates single-request latency

Ollama’s default `keep_alive` unloads an idle model after five minutes; the next question then pays a full reload from disk. On identical hardware and an identical question, a warm (resident) model answered in 13.46 s (6.96 s to first token); a cold (evicted) model took 263.44 s (225.69 s to first token) — a 19.6× slowdown. Because the reference deployment is lightly used outside peak hours, five minutes of quiet between questions is common, meaning most questions in an unoptimised deployment would pay this cost. Setting `OLLAMA_KEEP_ALIVE=-1` to keep all three models resident (at a cost of roughly 7.7 GB RAM) was, on its own, the single largest latency improvement made in this project — larger than the router and verification optimisations combined.

7.4.2 A cache alone does not survive simultaneous identical questions

Table 7.3 shows a 50-simulated-user load test drawing from 5 common questions. A semantic answer cache alone served only 2 of 50 requests, because every user’s request reached the cache lookup before the first answer had finished generating — a cache stampede that no similarity threshold can fix. Adding request coalescing, so that only the first caller of a given question generates an answer and every later identical request waits on that in-flight result, raised throughput roughly 7.7×; tuning the queue timeout on top of that let all 50 requests succeed.

Table 7.3: Effect of request coalescing under concurrent load (50 users, 5 questions)

Configuration	Served	Busy / failed	Throughput
Cache only	2	48	2.4/min
+ Request coalescing	21	29	18.4/min
+ Tuned queue timeout	50	0	28.1/min

7.5 Discussion

Two findings generalise beyond this specific deployment. First, in a multi-agent pipeline built around a single shared LLM runtime, the agent that looks cheapest to build — a router doing

a simple four-way classification — can dominate cost if implemented as a full LLM call, and is often the easiest agent to replace with a much cheaper method without a measurable accuracy loss, as Section 5.3 showed. Second, caching alone is not a sufficient concurrency strategy for an LLM-backed service; without request coalescing, a cache only helps requests that arrive strictly after the answer they want already exists, which is close to never true under simultaneous load.

7.6 Limitations

RAG-answer verification, at 62.3 seconds median after optimisation, remains the largest single remaining stage and the main target for further work. Synthesis time on CPU-only hardware is noisy enough ($\pm 30\%$ run-to-run) that single-run comparisons of synthesis latency specifically should not be over-interpreted; the router and verification results in this chapter are trusted because they moved by two to four orders of magnitude, well outside that noise band. All performance figures in this chapter are from one representative hardware configuration; behaviour on GPU-accelerated hardware was not measured and is identified as future work in Chapter 8.

Chapter 8

Conclusion and Future Scope

8.1 Conclusion

This project set out to answer whether a university could offer a conversational, natural-language interface to its live institutional data without sending that data to a cloud service and without granting a language model unsupervised write or broad read access to its database. The system built and evaluated in this report answers both questions affirmatively. All inference — routing, SQL generation, retrieval, synthesis, and verification — runs locally through Ollama; the database role available to the model is read-only and limited to 12 non-sensitive tables, independently enforced both by SQL parsing and by database role privileges; and the web-fetch path can never be pointed at an attacker-chosen or internal address. Measured on CPU-only commodity hardware, targeted optimisation of the router and verification stages cut end-to-end median latency for a seven-question benchmark by 40%, and a separate scaling study showed the system, once model residency and request coalescing were addressed, serving all 50 simulated concurrent users of a shared-question workload rather than the 2 it served initially. Taken together, the results support the project’s central claim: a defence-in-depth, multi-agent architecture is a practical way to give an institution conversational access to its own data without a cloud dependency and without trusting the language model itself with anything it should not have.

8.2 Future Scope

- **GPU-accelerated serving.** Since LLM inference accounts for 72.5% of pipeline time on CPU-only hardware, a GPU serving layer is the most direct remaining path to lower latency, and is a natural next deployment tier above the CPU baseline documented in this report.
- **Broader, still-guarded data access.** Additional read-only roles, scoped to specific staff or departmental needs, could extend coverage beyond the current 12-table allowlist without weakening the least-privilege guarantee.
- **Adaptive request coalescing.** The queue-timeout tuning in Section 7.4 was found empirically; an adaptive scheme that adjusts to observed load could remove the need for manual tuning.
- **Native mobile client.** The current interface is a responsive web client; a native mobile application would extend reach without changing the backend architecture.
- **Multi-institution deployment templates.** Packaging the allow-list, schema mapping, and Compose configuration as a per-institution template would reduce the effort required to deploy the system at a second institution.
- **Formal red-teaming of the verification agent.** The verification stage is currently evaluated on correctness and latency; a structured adversarial evaluation of how reliably it catches an incorrect or fabricated answer would strengthen the safety case made in Chapter 4.

Bibliography

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [2] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, “Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task,” *Proceedings of EMNLP*, 2018.
- [3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” *International Conference on Learning Representations (ICLR)*, 2023.
- [4] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, “LLaMA: Open and Efficient Foundation Language Models,” *arXiv preprint*, 2023.
- [5] Qwen Team, Alibaba Group, “Qwen2.5 Technical Report,” *arXiv preprint*, 2024.
- [6] Django Software Foundation, *Django Documentation*. <https://docs.djangoproject.com>.
- [7] PostgreSQL Global Development Group, *PostgreSQL Documentation*. <https://www.postgresql.org/docs/>.
- [8] Qdrant, *Qdrant Vector Database Documentation*. <https://qdrant.tech/documentation/>.
- [9] Ollama, *Ollama Documentation*. <https://ollama.com>.
- [10] Ministry of Electronics and Information Technology, Government of India, *Digital Personal Data Protection Act, 2023*.
- [11] OWASP Foundation, *OWASP Top 10:2021*. <https://owasp.org/Top10/>.